



APUNTES DE CLASE

11-08-26

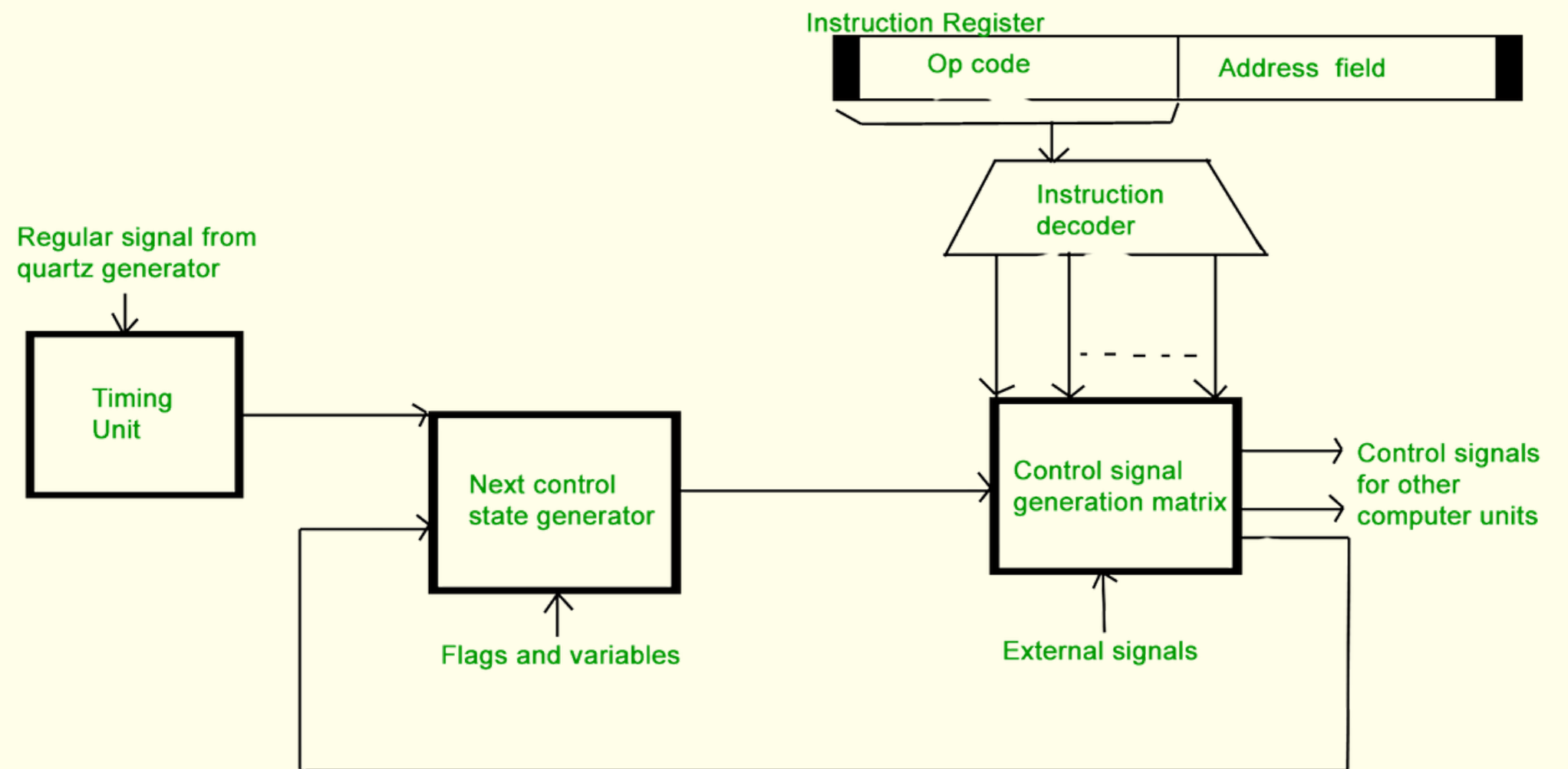
David Morales Vargas

[Compiladores e Interpretes]

Unidad de Control

Alambrada

- Creadas con compuertas lógicas.
- Diseño complejo de modificar.
- Cada instrucción en lenguaje máquina tiene un circuito físico.
- Rápidos y eficientes, pero poco flexibles.
- Adecuada cuando el conjunto de instrucciones es estable.

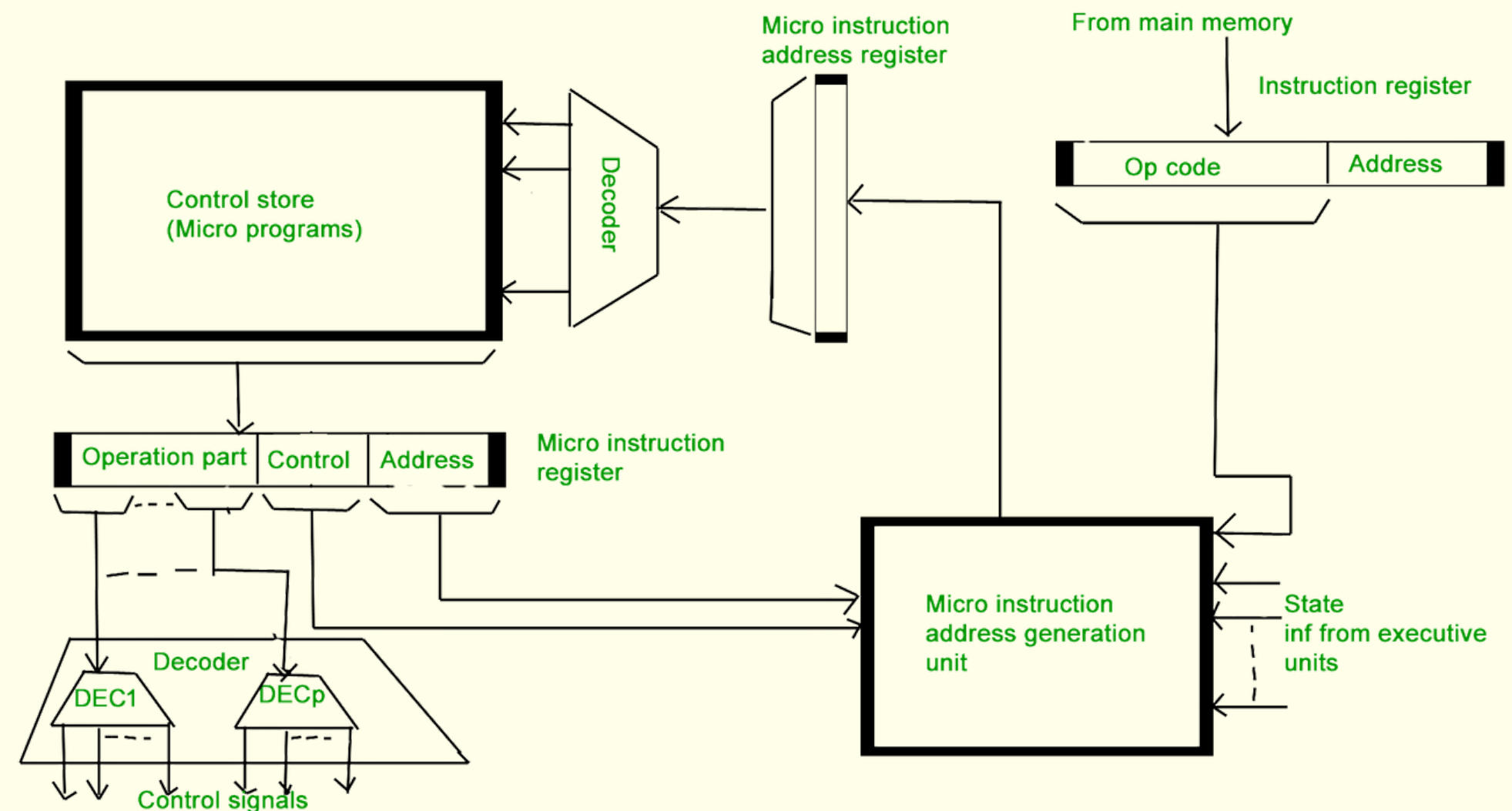


Unidad de Control

Microprogramada

TODO ES MICRO

- Más flexible y fácil de corregir.
- Puede ser más lenta por la lectura de **microinstrucciones**.
- Reduce la complejidad del hardware visible al diseñador.
- Propuestas por Wilkes en 1952.
- Cada instrucción de lenguaje máquina se construye con **micromúltiples microinstrucciones microtomadas** de una **micromemoria**.



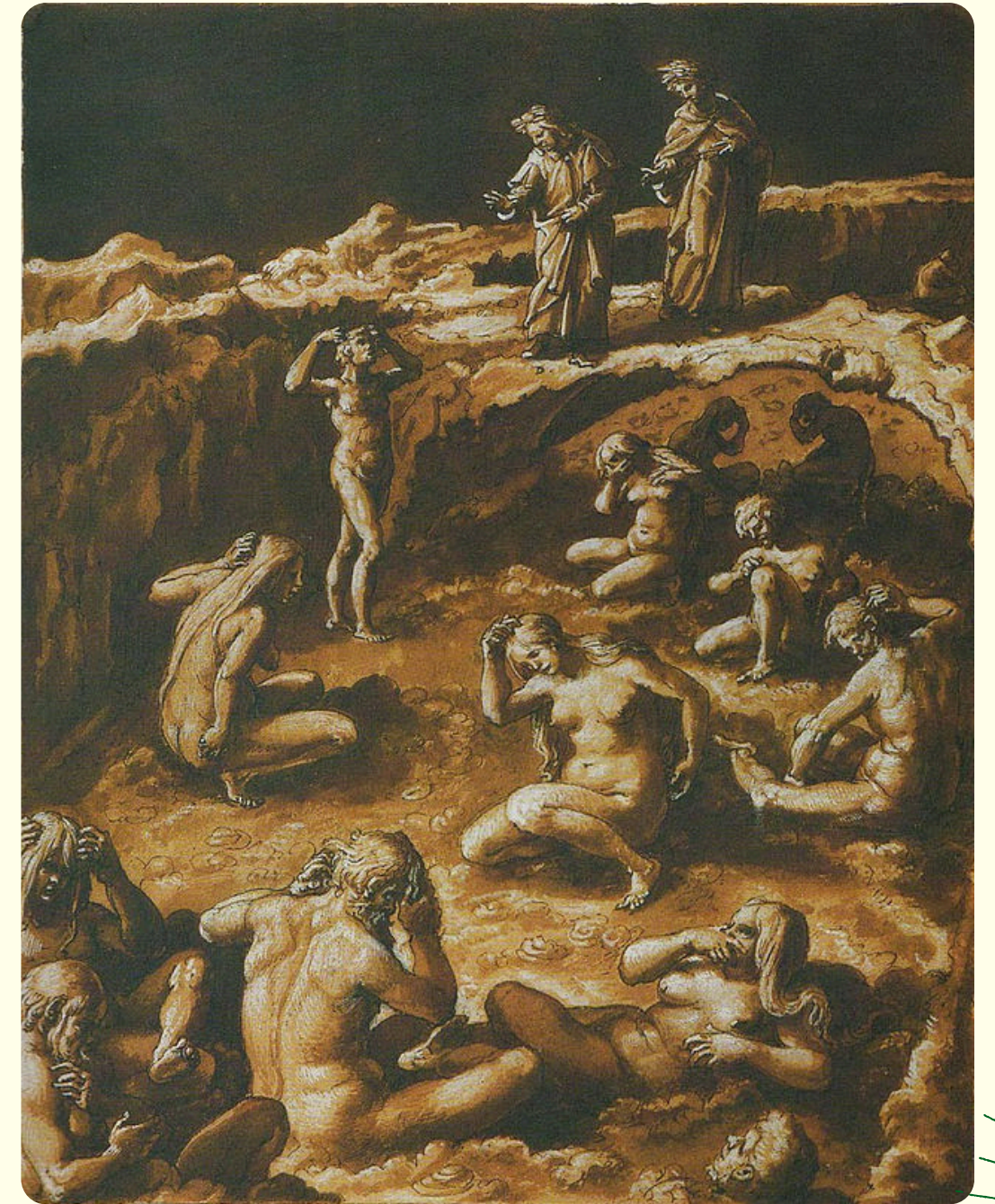
Aplicación Microprogramada

El diablo nos castiga en el infierno
haciendonos programar una aplicación de
contabilidad microprogramada.

Se duraria demasiados años haciendola y
tendría mucha brecha semántica pero sería
la contabilidad más eficiente.

* No encontré el chiste del infierno de la caca pero aquí hay uno
del infierno tico: [click aqui](#)

** Obsecuencia significa Sumisión, amabilidad, condescendencia.



Infierno de la obsecuencia o de al caca.



Productividad vs Eficiencia

Productividad: Líneas de código sin errores por unidad de tiempo al desarrollar un sistema.

Eficiencia: cantidad de recursos (tiempo, espacio, gasolina, energía) que un sistema en funcionamiento requiere para cada resultado.

Microprogramación:

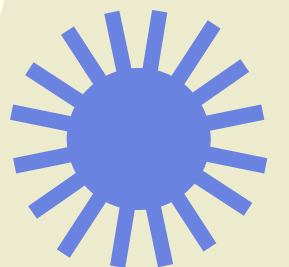
P

E

Capas:

Microprogramación

Hardware



Lenguaje Máquina

- Más fácil que hacer **micro**programación.
- Responsabilidades del programador:
 - Enviar el programa en RAM.
 - Inicializar el Program Counter.
 - Comenzar ejecución.
- Conocimientos que debía tener:
 - Saberse los códigos de operación.
 - Modos de direccionamiento.
 - Calcular tamaños y desplazamientos.
- NO era portátil ya que el lenguaje dependía de la computadora en la que se hiciera.

Arquitectura
≡
lenguaje máquina

Más Capas:

Lenguaje Máquina

Microprogramación

Hardware

P



E



Perdimos
eficiencia :(

Lenguaje Ensamblador

- Lo inventan en 1950.
- Es un programa que recibe mnemónicos con una sintaxis simple y genera el lenguaje máquina.
- Llega a facilitar la programación ya que calcula desplazamientos, evita tener que memorizar códigos de operación.
- Usa pseudoinstrucciones y macros.
- Más facil pero seguia sin ser portátil.

P



Ganamos
productividad y no
perdimos eficiencia
yeiii

E



Even moar Capas:

Lenguaje Ensamblador

Lenguaje Máquina

Microprogramación

Hardware

Casi no me veo :(

Lenguajes de Alto Nivel

- Por primera vez, era posible desarrollar programas sin depender directamente del hardware y lograr mayor portabilidad.
- Cada vez existe un mayor nivel de abstracción en la programación.
- El hardware se vuelve prácticamente irrelevante para el programador.
- Permite alcanzar una mayor productividad.

P

Perdimos
eficiencia
again :(

E

Todavía más más capas:

lenguajes de Alto Nivel

lenguaje Ensamblador

lenguaje Máquina

Microprogramación

Hardware

HELP!!!!!!

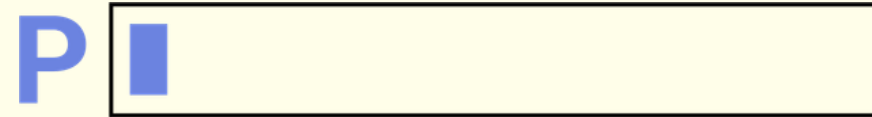
Productividad

vs

Eficiencia

Resumen

Microprogramación:



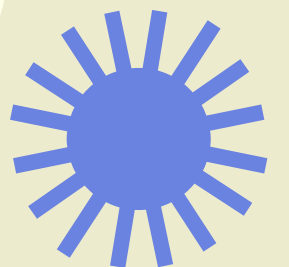
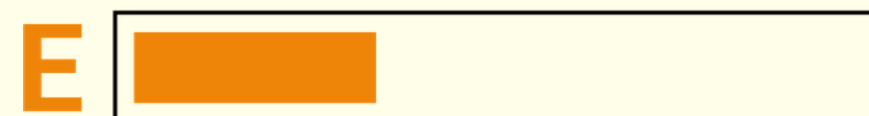
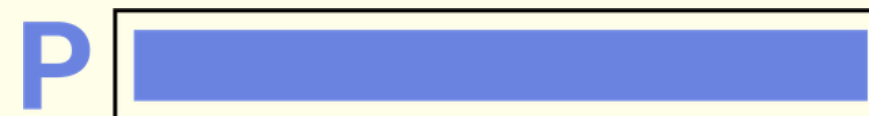
Lenguaje Máquina:



Lenguaje Ensamblador:



Lenguajes de Alto Nivel:



Sistemas Operativos

Nacimiento

Suena increíble pero hubo primero compiladores que sistemas operativos :o.

El nacimiento de estos chiquitos se remonta en los 50`s cuando se necesitaba reservar horas de máquina en una máquina dedicada.

Windows?
MacOS?
Linux?

Sistema operativo



Máquina Dedicada

Las universidades tenían sus mainframes las cuales podían usar cualquier estudiante/funcionario.

Para esto tenían que reservar “horas de máquina”, los hacía responsables de todo lo que pudiera pasar ahí dentro.

El Boot era manual, lento y propenso a errores.

4 segundos de
boot es lento? JA
JA JA



Máquina Dedicada Uso ideal

2 horas solo en
booteos.

1. Configurar los tableros de control.
2. Poner FORTRAN en el lector de tarjetas y bootear el computador.
3. Colocar nuestro código fuente FORTRAN en el lector, a veces esto llevaba varias pasadas.
4. El compilador perfora un programa equivalente en ensamblador.
5. Colocar ensamblador en el lector y volver a bootear.
6. Poner el código ensamblador generado anteriormente en el lector, otra vez quizás varias pasadas.
7. Perfora el equivalente en lenguaje máquina.
8. Poner tarjeta de lenguaje máquina recién creada en el lector, volver a bootear y aquí el programa corre.
9. Recoger los resultados de la impresora.
10. Repetir.

Máquina Dedicada Análisis

Ventajas:

- ★ El programador tenía el control absoluto.
- ★ Muy poca gente sabía cómo hacer todo esto, quienes lo sabían eran considerados expertos.
- ★ Era fácil controlar quién usaba el equipo.

Desventajas:

- ★ Desperdicio de recursos.
- ★ Repetición de trabajo entre los diferentes usuarios.
- ★ Era necesario conocer el hardware, ya que, en caso de un incidente, uno mismo debía solucionarlo.

TAREA MORAL

Leer “General Motors/North American Monitor for the IBM computer 704”
- Robert L. Patrick

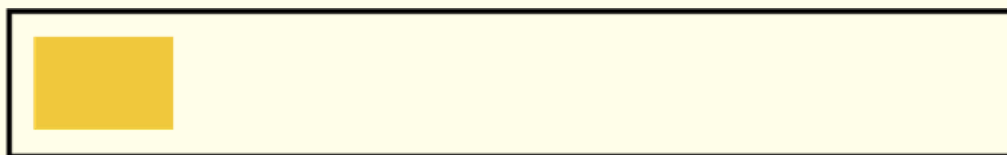
Máquina Dedicada Mejorandola

El sector administrativo dijo: que desmadre es estar en esta vara y crean el operador de computadoras.

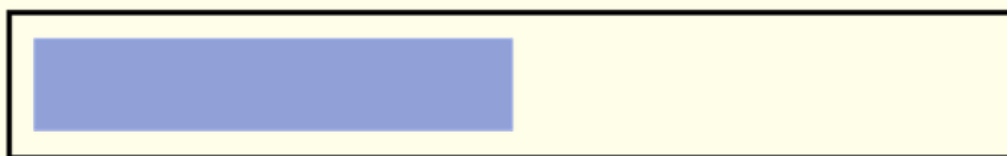
- Se recolectan trabajos y factorizan pasos.
- Configuraciones estandar.
- Aumenta el throughput del centro de computo.

Productividad del Centro de Computo:

Máquina
Dedicada



Batch con
Operador



¡FUERA
PROGRAMADORES!



Máquina Dedicada

Mejorandola aun más

El sector administrativo volvió hacer las suyas y se preguntaron: ¿Realmente es necesario un operador?.

- Su trabajo era el mismo siempre y casi mecánico.
- Compilar - Ejecutar - Generar Reporte - ↻
- El operador se dormía entre procesos.

Dijeron: ¿Y si hacemos un software que lo haga solo?

A baby is born

* Claramente no hicieron Windows xD



Sistemas Operativos Primitivos

Para este punto ya había muchos más compiladores, el hardware era más eficiente y fallaba menos y había más periféricos disponibles.

La idea administrativa era hacer un software que remplazara al operador (hasta cierto punto) con sus labores de seleccionar compilador, compilar, ejecutar y generar listado.

Nace el Operating System, mal traducido a Sistema Operativo por alguien desconocido.

- El SO corre siempre en memoria.

Spoiler: se vienen más tarjetas

Con este nacimiento nacen también varias preguntas:

¿Cómo va saber el S.O hasta donde llega cada trabajo?

¿Cómo sabe cual compilador usar?

¿Cómo evitar que mezcle trabajos?

Estas y otras preguntas serán contestadas en los apuntes de la próxima clase.

Dato extra

Trate de buscar quien tradujo por primera vez Operating System a español y fallé en la búsqueda, pero me encontré con esta noticia: [aquí](#).

Vaquero nos cuenta que, entre los años 60 y 80, las **traducciones de los libros de informática a la lengua de Cervantes eran “horrorosas”** y “para echarse a llorar”. “Las hacían en Argentina y en México gente que se veía que ni sabía español ni sabía informática”, asegura. Cuando el gran público comenzaba a conocer la existencia de ordenadores, que un traductor escogiese los términos informáticos sin entenderlos casi siempre acababa en desastre.

¿Un ejemplo? **'Array'**, un término que en informática alude a cierto tipo de estructura de datos. **Unos la traducían por el americanismo “arreglo”**, otros como “matriz” y los más puristas, como “formación”, el término que para este catedrático jubilado es el más correcto.

“Es estrictamente el mismo concepto que una formación cristalina, pero la palabra 'arreglo' es muy mala”, explica Vaquero. Cuando un alumno se topaba con “arreglo” en un libro, difícilmente podía comprender la lección.

Los españoles haciendo lo suyo siempre.